

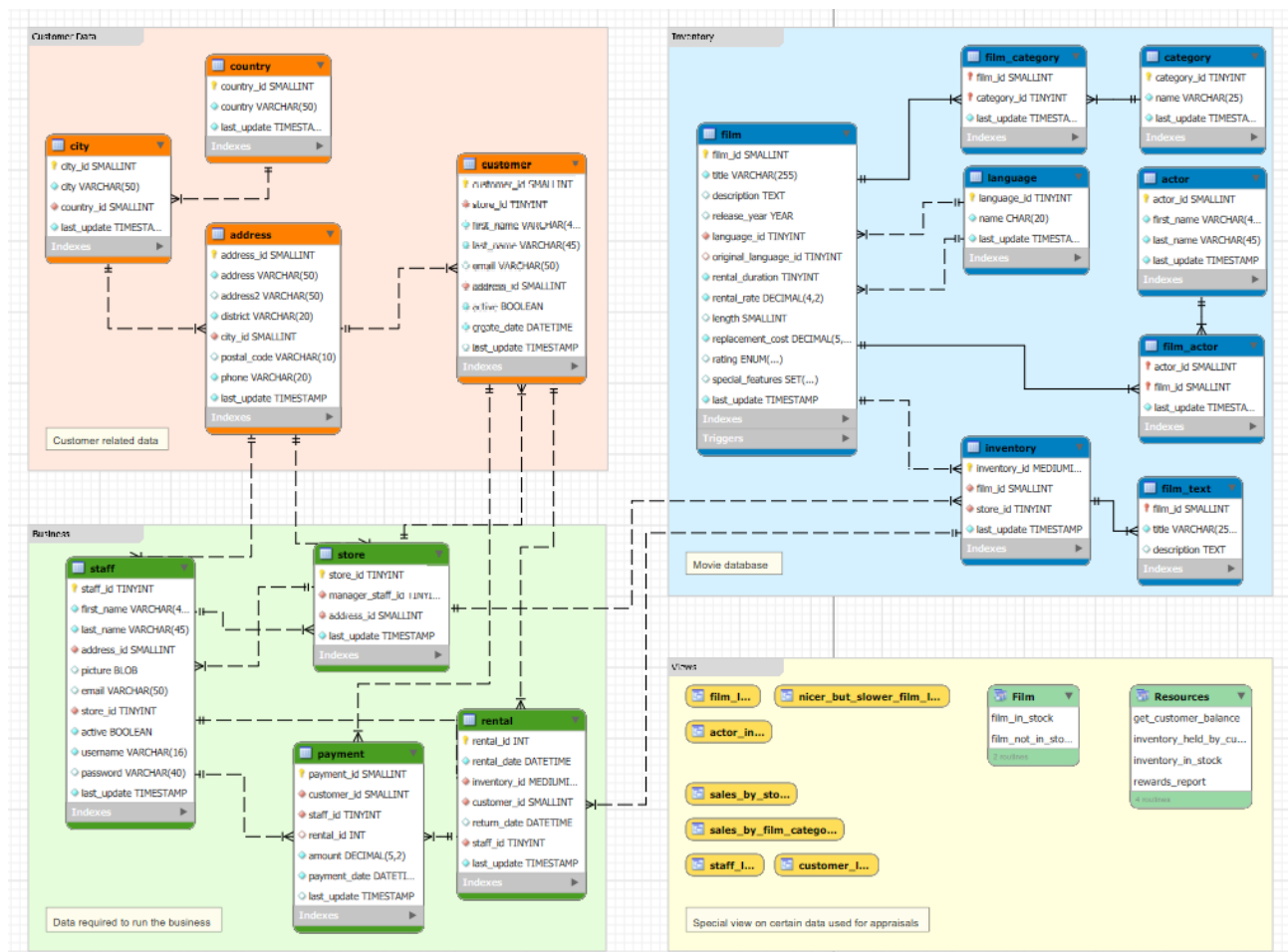
En este laboratorio, utilizaremos el esquema de bases de datos Sakila, disponible en MySQL. Este esquema fue desarrollado por Mike Hillyer en 2005. Representa una tienda online de arriendo de películas (DVD) y está basado en un modelo previo desarrollado por Dell.

Es interesante conocerlo, no sólo para practicar sentencias SQL, sino porque es utilizado hasta el día de hoy en documentación y foros de MySQL, entonces hay muchos ejemplos que han sido desarrollados utilizándolo como base. Eso permite que nos concentremos más en la herramienta y menos en la comprensión del modelo.

Para más información respecto del modelo, como por ejemplo su estructura y ejemplos, puede revisar <https://dev.mysql.com/doc/sakila/en/>, pero **se recomienda consultar a ChatGPT** en relación al contenido de cada tabla. Como **SAKILA** es un modelo muy utilizado, la información que entrega ChatGPT es clara y precisa. Ejemplo de prompt:

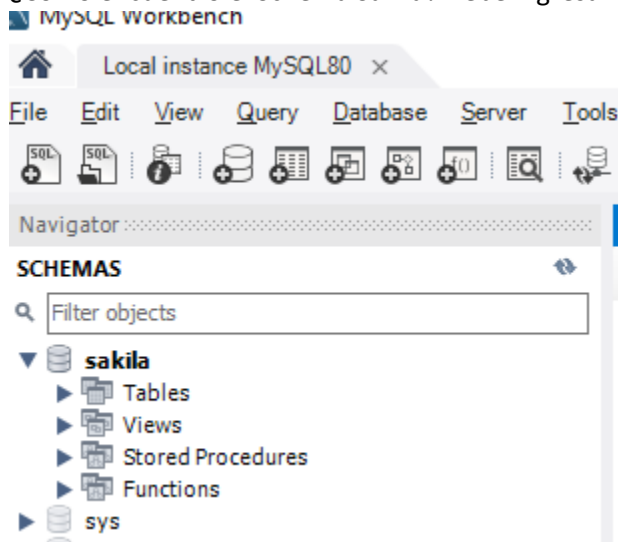
- Necesito utilizar la tabla FILM del esquema SAKILA, de MySQL. Por favor, describe qué contiene cada atributo.

Una panorámica del modelo es la siguiente. En él se aprecian las tablas y vistas (consultas almacenadas).



1. Comprendiendo el modelo de datos:

- a) ¿Cómo encuentro el Schema Sakila? Debe ingresar a MySQL Workbench, a localhost (o equivalente).



- b) ¿Cómo se relacionan las tablas actor y film_actor? (1-N, N-M, N-1). Para ello, revise el diagrama y también las llaves primarias y foráneas de ambas tablas. ¿Cuál tabla tiene la llave foránea?
- c) ¿Qué significa, entonces, ese símbolo? ¿Qué podemos decir respecto de la relación entre las tablas customer y payment, sólo mirando el diagrama?
- d) ¿En qué tabla se encuentra el listado de películas? ¿Qué atributos (datos) tenemos de cada película?
- e) Escriba una consulta que entregue el listado de películas que tiene una duración mayor a 90 minutos.
- f) ¿Cómo podemos saber en qué idioma está la película "AGENT TRUMAN"?
- g) ¿Cómo podemos saber a qué categoría(s) pertenece la película "AGENT TRUMAN"?

2. Consultas que involucran a varias tablas (joins).

Una consulta involucra a varias tablas cuando necesito obtener información de más de una tabla, o cuando necesito aplicar filtros que involucran a otras tablas. Ejemplo:

Obtener el nombre de una película y el idioma.

```
select f.title pelicula, l.name idioma
from sakila.film f, sakila.language l
where f.language_id = l.language_id
```

Observen que estoy haciendo un **join** entre las tablas film y language. Quizás, ustedes no lo vieron de esa manera, sino de esta:

```
select f.title pelicula, l.name idioma
from sakila.film f join sakila.language l
on f.language_id = l.language_id
```

(*) Observe el uso de alias, tanto para las tablas como para los nombres de atributos. ¿Con qué propósito creen que lo hago?

Desarrolle a continuación los siguientes ejercicios:

- a) La columna length de la tabla film contiene la duración (en minutos) de cada película. Escriba una consulta que tenga las siguientes columnas: title, release_year, language.name, category.name, para todas las películas con duración superior a 90 minutos. Antes de escribir la consulta, identifique las tablas necesarias.
- b) Supongamos que tiene el identificador de un cliente (customer_id). Por ejemplo, el n° 3 (Linda Williams). ¿Qué tablas debo utilizar para obtener una lista con los nombres de las películas que ella ha rentado? Haga una consulta que entregue la lista de películas rentadas por ella, indicando su título, language.name, fecha de arriendo y fecha de devolución.
- c) Para la misma cliente de la pregunta anterior, escriba una consulta que entregue el nombre y apellido de los empleados que le han rentado películas a ella. HINT: utilice “select distinct” para que los nombres no se repitan, en caso que el mismo empleado le haya rentado varias películas.

3. Uso de funciones de grupo.

Las funciones de grupo nos permiten realizar operaciones que involucran a varios registros (filas), retornando un solo valor para ese conjunto de registros. Las funciones de grupo más comunes son count, max, min, avg (promedio), sum.

El siguiente ejemplo, nos muestra cuántas películas ha arrendado Linda Williams.

```
select count(*) from rental where customer_id = 3
```

HINT: Count(*) cuenta la cantidad de registros.

No siempre queremos calcular un único total. Lo más usual es que queramos agrupar los registros, y calcular totales por grupos de registros, dado algún criterio. Por ejemplo, podemos querer saber cuántas películas ha arrendado cada cliente:

```
select c.first_name, c.last_name, count(*)  
from sakila.customer c join sakila.rental r  
on c.customer_id = r.customer_id  
group by c.first_name, c.last_name
```

Desarrolle a continuación los siguientes ejercicios:

- ¿En qué tabla, y en qué atributo, encontramos cuánto ha pagado cada cliente por arriendo de películas?
- Entregue un listado que contenga el nombre y apellido del cliente, indicando el monto total que ha pagado por arriendos cada cliente.
- Imagine que debe pagar un bono a cada empleado, correspondiente al 5% del monto de los arriendos que ese empleado ha cobrado. Entregue un listado con el nombre y apellido de los empleados, el monto total de pagos recibidos por empleado, y el cálculo del bono propuesto para cada empleado.

4. Ordenamiento.

Es usual que necesitemos generar una lista ordenada de registros y que estos no se encuentren necesariamente almacenados de forma ordenada. Para ello, podemos utilizar la cláusula ORDER BY. En el ejemplo anterior, podemos ordenar a los clientes, mostrando primero a quienes han arrendado más películas:

```
select c.first_name, c.last_name, count(*) cantidad  
from sakila.customer c join sakila.rental r  
on c.customer_id = r.customer_id  
group by c.first_name, c.last_name  
order by cantidad desc
```

Desarrolle a continuación los siguientes ejercicios:

- Entregue un listado que contenga el nombre y apellido del cliente, indicando el monto total que ha pagado por arriendos cada cliente. El listado debe ordenarse alfabéticamente por apellido y nombre de clientes.
- Entregue un listado que contenga el nombre y apellido del cliente, indicando el monto total que ha pagado por arriendos cada cliente. El listado debe ordenarse según el monto total pagado por arriendo por cada cliente, de manera ascendente.

5. Funciones de fecha.

MySQL utiliza el tipo de datos `datetime` para almacenar fechas. Esta fecha puede incluir, además del día, mes y año, horas, minutos y segundos. Podemos calcular el tiempo transcurrido entre 2 fechas con mucha precisión, pero a veces queremos obtener una parte de la fecha, por ejemplo, el día, el mes o el año. Veamos un ejemplo:

```
select year (rental_date), month (rental_date), day (rental_date)
from sakila.rental
```

Desarrolle a continuación los siguientes ejercicios:

- a) Cantidad y monto total de pagos recibidos por empleado, considerando sólo el mes de mayo de 2005. Debe entregar el nombre y apellido de cada empleado y ordenar el listado por apellido del empleado.
- b) Una función bastante útil es **DateDiff**, que entrega la cantidad de días transcurridos entre 2 fechas. Genere un listado que entregue, ordenados por fecha de arriendo, el nombre y apellido del cliente, el monto del arriendo, la fecha de arriendo, la fecha de devolución, y la cantidad de días transcurridos.
- c) Aplique un filtro al listado anterior, mostrando sólo aquellos arriendos iguales o superiores a 5 días.

6. Subconsultas.

Hablamos de subconsultas, o consultas anidadas, cuando el resultado de una consulta es utilizado como entrada para otra consulta. Existen diferentes razones para utilizar subconsultas, entre ellas:

- Hacer el código más legible.
- Utilizar una subconsulta en un filtro, no para desplegar esos datos.
- En una consulta de cierta complejidad, utilizar el resultado de una consulta como una “tabla intermedia” (en álgebra relacional, tablas y resultados de consultas son relaciones, y por lo tanto para el SQL son equivalentes).

Veamos un ejemplo. Queremos obtener el listado de películas que fueron rentadas más de una vez en el mes de mayo de 2005. Nos interesa obtener el listado de las películas, no queremos retornar datos relativos a los arriendos.

Esta consulta nos dice cuántas veces fue rentada cada película (ejm: film_id=1) en el mes de mayo de 2005:

```
select count(*) from inventory i join rental r
  on i.inventory_id = r.inventory_id
 where
    i.film_id = 1 and
    year(r.rental_date) = 2005 and month(r.rental_date) = 5
```

Podemos usar esa consulta en un filtro aplicado a la lista de films:

```
select * from film f
 where
    1 < (
      select count(*) from inventory i join rental r
        on i.inventory_id = r.inventory_id
       where
          f.film_id= i.film_id and
          year(r.rental_date) = 2005 and month(r.rental_date) = 5
    )
```

Desarrolle a continuación los siguientes ejercicios:

- Realice una consulta que entregue, para el mes de mayo de 2005, el total de pagos recibidos por cada tienda (store).
- Agregue un filtro a la consulta anterior, de modo que el total de pagos recibidos considere sólo las películas que son de categoría “Children”.